DAY 1

LLM Fundamentals and API Interaction

Agentic AI Learning Roadmap · Theory Notes

A structured lesson covering LLMs, API fundamentals, prompting, tokens, context windows, conversation history, memory, and responsible API use.

Learning objective

Understand how a Python application communicates with a large language model, how the request is structured, how the model uses context, and what an application must do to manage conversations and failures.

Today's Syllabus

- 1. How LLM applications work
- 2. API fundamentals: HTTP, endpoints, authentication, JSON, SDKs, and errors
- 3. Messages and prompting: roles, instructions, context, constraints, and output requirements
- 4. Tokens and context windows: input/output usage, limits, and cost
- 5. Conversation history and memory: current context, history, persistent memory, and RAG
- 6. Reliability and responsible API use: credentials, retries, usage tracking, and validation

1. How an LLM Application Works

An LLM, an API, an API client, and an AI application are related but different components.

LLM (Large Language Model): A neural network trained to process and generate language. At inference time, it uses the input context to generate a sequence of output tokens. It does not simply retrieve a prewritten answer from a database.

Training vs. inference

- **Training:** Model parameters are adjusted using training data.
- **Inference:** A trained model generates an output in response to an input. Application developers usually work with inference rather than training a model from scratch.

API: An Application Programming Interface defines how one software component interacts with another. A remote LLM API lets your program send requests to a provider and receive responses without necessarily running the model locally.

API client / SDK: A client or Software Development Kit helps your code communicate with the API. An SDK can handle request construction, authentication details, and response parsing.

AI application: The model is only one part of the product. The application accepts input, prepares instructions and context, may retrieve data or call tools, sends a model request, checks or processes the result, and returns an answer.

Core idea: the LLM generates language, while your application controls the surrounding workflow and decides what to do with the output.

2. API Fundamentals

HTTP request and response

An application commonly communicates with a remote LLM using HTTP. A request includes a method and endpoint, headers, and often a body. The response includes a status, metadata, and a body containing generated content and possibly usage information.

Request part	Purpose
Method and endpoint	Specify the operation and where it is sent.
Headers	Carry authentication, content type, and other metadata.
Body / payload	Contains model selection, messages/input, and supported parameters.
Response	Returns status, generated content, and sometimes usage metadata.

Endpoint: The URL for an API operation. **HTTP method:** The type of operation; POST is commonly used to submit generation requests. **JSON:** A data format that represents objects, arrays, strings, numbers, booleans, and null. Exact fields and formats depend on the provider.

Illustrative JSON payload (not a universal schema):

```
{
  "model": "MODEL_NAME",
  "input": "Explain binary search."
}
```

Authentication and API keys

An API key is a credential used to authorize API access. Store it in an environment variable or secrets manager; never commit it to GitHub, expose it in frontend code, or print it in logs. For a web product, a common pattern is for the frontend to call your backend, which then calls the provider using a private key.

SDK vs. raw HTTP

Raw HTTP gives you direct control over request construction and response parsing. A provider SDK wraps common operations and reduces boilerplate. Even when using an SDK, understanding HTTP and JSON helps with debugging.

Common API error categories

- **Authentication:** Check the key, permissions, and configuration.
- **Rate limiting:** Slow down and retry with backoff when appropriate.
- **Invalid request:** Correct unsupported parameters or malformed input.
- **Network or server failure:** Retry safely when appropriate.
- **Quota or billing:** Check usage limits and account settings.

Use bounded retries and backoff. Do not retry every error indefinitely; repeated requests can waste money or worsen rate limits.

3. Messages and Prompting

A prompt is the input or set of instructions supplied to a model. It can include a question, instructions, background information, examples, constraints, required output format, and conversation history.

A weak prompt such as “Explain SQL joins” gives little guidance. A stronger prompt specifies the audience, purpose, comparisons, examples, and expected structure. Clear prompts improve the chance of a useful response but cannot guarantee correctness.

Common message roles

- **System:** High-level behavioral guidance in APIs that support this role.
- **Developer:** Application-level instructions and rules.
- **User:** Input from the person using the application.
- **Assistant:** Model-generated messages.
- **Tool:** Results returned by a tool in APIs that represent tool results this way.

Roles and formats differ across providers. Some APIs combine or represent instructions differently, so do not assume every API uses the same structure.

Instructions vs. context

- **Instruction:** “Summarize this document in five bullet points.”
- **Context:** The document text itself.
- **Constraint:** “Do not include claims unsupported by the document.”
- **Output requirement:** “Return five concise bullets.”

A practical prompt checklist

- Task: What should the model do?
- Context: What information does it need?
- Constraints: What must it avoid or preserve?
- Output: What should the result look like?

Prompts cannot guarantee truth. For higher-reliability systems, use trusted retrieval, structured-output validation, tool-based verification, evaluations, or human review as appropriate.

4. Tokens and Context Windows

Tokens are units used to represent text. A token can be a word, part of a word, punctuation, or another text unit. Tokens do not map exactly to words, and tokenization differs between models.

Input tokens include the instructions, messages, and other content supplied to the model. **Output tokens** are generated by the model. Some providers also report cached input, reasoning, or other usage categories.

Context window is the amount of information a model can process within a request, subject to its limits and API rules. Long conversation histories can increase cost and latency, make information harder to manage, and eventually exceed limits.

A context window is not the same as persistent memory. A model may process information for the current request without permanently retaining it for future requests.

Output limits can cause an answer to stop before completion. When troubleshooting incomplete output, check whether the model reached a token limit before assuming the prompt was wrong.

Pricing varies by provider and model. Some APIs price input and output tokens differently and may have additional categories. Inspect returned usage metadata when available.

5. Conversation History and Memory

Many LLM API interactions are stateless from the application's perspective: a new request does not necessarily include prior requests. In a simple stateless pattern, your application sends the relevant history again. Some APIs provide conversation-management features, so always check the behavior of the specific API.

Three concepts to distinguish

- **Current request context:** Information supplied for one request, such as instructions, the latest question, and relevant material.
- **Conversation history:** Messages retained by your application or conversation feature so earlier turns can inform later answers.
- **Persistent memory:** Selected information stored beyond the immediate conversation and retrieved when relevant.

History may become too long to send in full. Persistent memory raises design questions about what to save, retrieve, update, and delete.

RAG (Retrieval-Augmented Generation) retrieves relevant information from an external knowledge source and supplies it as context. For example, a support assistant might retrieve a company's return policy before answering. RAG is not automatically memory, though a system can combine retrieval, conversation history, and persistent memory.

6. Reliability and Responsible API Use

Protect credentials

- Keep API keys out of source code, public repositories, browser-side code, and logs.
- Use environment variables or an appropriate secrets manager.
- If a key leaks, revoke or rotate it and check for unexpected usage.

Handle failures

- Distinguish invalid input, authentication errors, rate limits, network problems, and provider-side failures.
- Show users clear, non-sensitive error messages and log enough information to debug.
- Retry only when appropriate, with limits and backoff.

Track usage

Inspect returned usage metadata where available. Monitor latency, failure rates, and usage as the application grows. Long histories can cause unnecessary input-token usage.

Treat model output as untrusted

A model response is not automatically valid, safe, or correct. Validate required formats, enforce permissions, and use confirmation or independent checks for important actions. Drafting an email is less risky than sending it to customers automatically.

Core rule: the model generates outputs; your application decides what it is allowed to do with them.

Official Documentation for Day 1

API Quickstart — https://developers.openai.com/api/docs/quickstart

Authentication, SDK setup, making a request, and reading the response.

API Concepts — https://developers.openai.com/api/docs/concepts

Model, request, response, and API terminology.

Prompt Engineering — https://developers.openai.com/api/docs/guides/prompt-engineering

Instructions, context, and output requirements.

Token Counting — https://developers.openai.com/api/docs/guides/token-counting

Understanding and estimating token usage.

API Pricing — https://platform.openai.com/pricing

Check current pricing and usage categories.

Knowledge Check

1. What is the difference between an LLM and an LLM API?

The LLM generates outputs; the API provides an interface for accessing model functionality.

2. Why send conversation history with a new message?

To provide earlier context that may not be included automatically.

3. What is the difference between instructions and context?

Instructions tell the model what to do; context supplies relevant information.

4. What can happen when a long conversation history is repeatedly sent?

Input usage and latency can grow, and the request may eventually exceed context limits.

5. Should an application blindly execute every action suggested by an LLM?

No. The application must enforce permissions and validate actions.

Day 1 Completion Goal

Before moving to implementation, aim to explain the LLM/API distinction, describe request and response structure, write a clear prompt, explain tokens and context windows, distinguish conversation history from persistent memory and RAG, and describe basic credential and error-handling practices.

Implementation is intentionally excluded from these notes. Begin coding only when you are ready for the next part of the lesson.